📘 Assignment: Java JDBC & Database Programming (With Definitions & Examples)

🔷 1. Introduction to JDBC

📖 Definition:

JDBC (Java Database Connectivity) is an API in Java that allows Java applications to interact with databases (like MySQL, Oracle, etc.) by executing SQL queries.

💡 Importance:

- Enables Java to communicate with databases
- Used for storing, retrieving, and updating data
- Platform-independent

🏗️ JDBC Architecture:

- **DriverManager** → Manages database drivers
- **Driver** → Connects Java to DB
- **Connection** → Session with database
- **Statement** → Executes SQL queries
- **ResultSet** → Stores query results

💻 Example:

```
import java.sql.*;

public class JdbcExample {
   public static void main(String[] args) throws Exception {
      Class.forName("com.mysql.cj.jdbc.Driver");

      Connection con = DriverManager.getConnection(
         "jdbc:mysql://localhost:3306/test", "root", "password");

      Statement stmt = con.createStatement();
      ResultSet rs = stmt.executeQuery("SELECT * FROM student");

      while (rs.next()) {
```

```
            System.out.println(rs.getInt(1) + " " + rs.getString(2));
        }

        con.close();
    }
}
```

🔷 2. JDBC Driver Types

📖 Definition:

JDBC drivers are software components that enable Java applications to connect to databases.

🗂️ Types:

1. Type 1: JDBC-ODBC Bridge

- Converts JDBC calls into ODBC calls
- ❌ Slow and outdated

2. Type 2: Native API Driver

- Uses database-specific libraries
- Faster than Type 1

3. Type 3: Network Protocol Driver

- Uses middleware server
- Platform-independent

4. Type 4: Thin Driver

- Direct communication with DB
- ✅ Most commonly used (MySQL uses this)

💻 Example:

```
Class.forName("com.mysql.cj.jdbc.Driver"); // Type 4 driver
```

🔷 3. Steps to Create JDBC Connection

📖 **Definition:**

A sequence of steps to connect Java application with database.

📌 **Steps:**

1. Import packages
2. Load driver
3. Establish connection
4. Create statement
5. Execute query
6. Process results
7. Close connection

💻 **Example:**

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/test", "root", "password");

System.out.println("Connection Successful!");
```

🔷 4. Types of JDBC Statements

📖 **Definition:**

Statements are used to execute SQL queries.

📚 **Types:**

1. Statement

- Simple SQL queries

```
Statement stmt = con.createStatement();
stmt.executeUpdate("INSERT INTO student VALUES(1,'John')");
```

2. PreparedStatement

- Precompiled, secure (prevents SQL injection)

```
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO student VALUES (?, ?)");
ps.setInt(1, 2);
ps.setString(2, "Alice");
ps.executeUpdate();
```

3. CallableStatement

- Used for stored procedures

```
CallableStatement cs = con.prepareCall("{call getStudent(?)}");
cs.setInt(1, 1);
cs.execute();
```

🔷 5. CRUD Operations

📖 Definition:

CRUD = Create, Read, Update, Delete operations in database.

💻 Examples:

Insert:

```
stmt.executeUpdate("INSERT INTO student VALUES(3,'Raj')");
```

Update:

```
stmt.executeUpdate("UPDATE student SET name='Ravi' WHERE id=3");
```

Select:

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

Delete:

```
stmt.executeUpdate("DELETE FROM student WHERE id=3");
```

🔷 6. ResultSet Interface

📖 Definition:

ResultSet is an object that holds data returned from SQL query.

📌 **Methods:**

- next() → move forward
- previous() → move backward
- first() → first row
- last() → last row

💻 **Example:**

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");

while (rs.next()) {
   System.out.println(rs.getInt("id") + " " + rs.getString("name"));
}
```

🔷 **7. DatabaseMetaData**

📖 **Definition:**

Provides information about the database.

📌 **Uses:**

- DB name
- Version
- Tables

💻 **Example:**

```
DatabaseMetaData dbmd = con.getMetaData();

System.out.println("DB Name: " + dbmd.getDatabaseProductName());
System.out.println("DB Version: " + dbmd.getDatabaseProductVersion());
```

🔷 **8. ResultSetMetaData**

📖 **Definition:**

Provides information about columns in ResultSet.

📌 **Methods:**

- getColumnCount()
- getColumnName()
- getColumnType()

💻 **Example:**

```
ResultSetMetaData rsmd = rs.getMetaData();

int count = rsmd.getColumnCount();

for (int i = 1; i <= count; i++) {
   System.out.println(rsmd.getColumnName(i));
}
```

🔷 9. SQL Query Examples

💻 **SQL Examples:**

Insert:

```
INSERT INTO student VALUES (1, 'John');
```

Update:

```
UPDATE student SET name='Alice' WHERE id=1;
```

Select:

```
SELECT * FROM student WHERE id=1;
```

Delete:

```
DELETE FROM student WHERE id=1;
```

🔷 10. Swing GUI with JDBC

📖 **Definition:**

Swing is a Java framework for creating GUI applications.

💻 **Example:**

```
JButton btn = new JButton("Insert");

btn.addActionListener(e -> {
    try {
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO student VALUES (?, ?)");
        ps.setInt(1, 1);
        ps.setString(2, "Test");
        ps.executeUpdate();
    } catch (Exception ex) {
        ex.printStackTrace();
    }
});
```

🔷 11. CallableStatement (Stored Procedure)

📖 **Definition:**

Used to call stored procedures from Java.

💻 **SQL Stored Procedure:**

```
CREATE PROCEDURE getName(IN empId INT, OUT empName VARCHAR(50))
BEGIN
    SELECT name INTO empName FROM employee WHERE id = empId;
END;
```

💻 **Java Example:**

```
CallableStatement cs = con.prepareCall("{call getName(?, ?)}");

cs.setInt(1, 1);
cs.registerOutParameter(2, Types.VARCHAR);

cs.execute();

System.out.println("Employee Name: " + cs.getString(2));
```